NLP:

Practical No 1:

Name: Karthik Vinod

Roll No: TCS2627033

Aim: To create a simple chatbot using a Python dictionary that responds to predefined user queries.

Code:

```
[13]: def chatbot():
          print("Bot: Hi!! I am a Simple Chatbot, Type 'Bye' to exit.")
          response={
              "hi" :"Hello, How can I help you?",
              "how are you?":"I am doing great, hope you are fine!!",
              "what is your name?":"I am chatbot 3.0",
              "where are you from?":"I am from Mumbai",
              "bye":"Goodbye! Have a nice day.."
          }
          print("Here are per-defined questions you may ask:")
          i=1
          for key in response.keys():
              print(f'{i}:{key}')
              i+=1
          while True:
              user_input=input("You: ").lower()
              if user_input== 'bye':
                  print(f"Bot: {response['bye']}")
                  break
              resp=response.get(user_input,"I am Sorry I do not understand the question.")
              print(f'Bot: {resp}')

      print("Karthik Vinod, TCS2627033")
      chatbot()
```

Output:

```
Karthik Vinod, TCS2627033
Bot: Hi!! I am a Simple Chatbot, Type 'Bye' to exit.
Here are per-defined questions you may ask:
1:hi
2:how are you?
3:what is your name?
4:where are you from?
5:bye
You:  hi
Bot: Hello, How can I help you?
You:  how are you?
Bot: I am doing great, hope you are fine!!
You:  what is your name?
Bot: I am chatbot 3.0
You:  where are you from?
Bot: I am from Mumbai
You:  bye
Bot: Goodbye! Have a nice day..
```

NLP:

Practical No 2:

Name: Karthik Vinod

Roll No: TCS2627033

Aim: Implement Sentence Segmentation and Work Tokenization.

Python Libraries & Modules:

- **pip install nltk:** Installs the Natural Language Toolkit (NLTK) library, which is used for Natural Language Processing (NLP) tasks in Python.
- **import nltk:** Imports the NLTK library so that its NLP functions can be used in the program.

NLTK Data Downloads:

- **nltk.download('punkt'):** Downloads the Punkt tokenizer dataset, which is used for sentence and word tokenization.
- **nltk.download('punkt_tab'):** Downloads the updated Punkt tokenizer data required by newer versions of NLTK for accurately identifying sentence and word boundaries.

Tokenization Functions:

- **from nltk.tokenize import sent_tokenize, word_tokenize:** Imports two tokenization functions:
- **sent_tokenize()** – Splits a paragraph into individual sentences.
- **word_tokenize()** – Splits text into individual words or tokens.
- **text = "This is simple text! Mr.XYZ wants it to be tokenized. It should be ready by 9.10 a.m.":** Stores the sample text that will be processed by the program.
- **sentences = sent_tokenize(text):** Performs sentence segmentation, dividing the given text into separate sentences.
- **words = word_tokenize(text):** Performs word tokenization, dividing the text into individual words and punctuation tokens.
- **print(f"Sentences: {sentences}"):** Displays the list of sentences obtained after sentence segmentation.
- **print(f"Words: {words}"):** Displays the individual words and tokens obtained after word tokenization.

Code:

```
[1]: pip install nltk
```

```
Collecting nltk
  Downloading nltk-3.9.4-py3-none-any.whl.metadata (3.2 kB)
Requirement already satisfied: click in c:\users\karthik\appdata\local\programs\python\python312\lib\site-packages (from nltk) (8.1.8)
Requirement already satisfied: joblib in c:\users\karthik\appdata\local\programs\python\python312\lib\site-packages (from nltk) (1.5.3)
Collecting regex>=2021.8.3 (from nltk)
  Downloading regex-2026.5.9-cp312-cp312-win_amd64.whl.metadata (41 kB)
Requirement already satisfied: tqdm in c:\users\karthik\appdata\local\programs\python\python312\lib\site-packages (from nltk) (4.67.1)
Requirement already satisfied: colorama in c:\users\karthik\appdata\local\programs\python\python312\lib\site-packages (from click->nltk) (0.4.6)
Downloading nltk-3.9.4-py3-none-any.whl (1.6 MB)
   ---------------------------------------- 0.0/1.6 MB ? eta -:--:--
   ------------ --------------------------- 0.5/1.6 MB 2.8 MB/s eta 0:00:01
   ---------------------------------- ------ 1.3/1.6 MB 4.5 MB/s eta 0:00:01
   ---------------------------------------- 1.6/1.6 MB 3.9 MB/s eta 0:00:00
Downloading regex-2026.5.9-cp312-cp312-win_amd64.whl (277 kB)
Installing collected packages: regex, nltk
Successfully installed nltk-3.9.4 regex-2026.5.9
Note: you may need to restart the kernel to use updated packages.
```

```
[2]: # Implement Sentence Segmentation and Work Tokenization
import nltk
nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\Karthik\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping tokenizers\punkt.zip.
```

```
[2]: True
```

```
[3]: import nltk
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\Karthik\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping tokenizers\punkt_tab.zip.
```

```
[3]: True
```

```
[14]: from nltk.tokenize import sent_tokenize, word_tokenize
text= "This is simple text! Mr.XYZ wants it to be tokenized. It should be ready by 9.10 a.m."
sentences=sent_tokenize(text)
words=word_tokenize(text)
print("Karthik Vinod, TCS2627033")
print(f'Sentences: {sentences}')
print(f'Words:  {words}')
```

Output:

```
Karthik Vinod, TCS2627033
Sentences: ['This is simple text!', 'Mr.XYZ wants it to be tokenized.', 'It should be ready by 9.10 a.m.']
Words:  ['This', 'is', 'simple', 'text', '!', 'Mr.XYZ', 'wants', 'it', 'to', 'be', 'tokenized', '.', 'It', 'should', 'be', 'ready', 'by', '9.10', 'a.m',
'.']
```

NLP:

Practical No 3:

Name: **Karthik Vinod**

Roll No: **TCS2627033**

Aim: Program to perform text processing which involves text normalization, lower case Converter, tokenization, stop words removal, punctuation removal, text correction, stemming and Lemmatization.

Definition:

Term	Simple Definition	Example
Text Normalization	Converting text into a standard and consistent format.	"Hello!!!" → "hello"
Lowercase Converter	Changes all letters to lowercase.	"HELLO World" → "hello world"
Tokenization	Splitting text into individual words or tokens.	"I love AI" → ["I", "love", "AI"]
Stop Words Removal	Removing common words that add little meaning.	"I am going to school" → "going school"
Punctuation Removal	Removing punctuation marks from text.	"Hi, John!" → "Hi John"
Text Correction	Correcting spelling or typing mistakes.	"teh" → "the"
Stemming	Reducing words to their root form by removing endings.	"playing", "played" → "play"
Lemmatization	Converting words to their dictionary (base) form using meaning.	"better" → "good", "running" → "run"

Description of Imported Python Libraries and Modules:

import nltk:

- Imports the Natural Language Toolkit (NLTK), a Python library used for Natural Language Processing (NLP) tasks.

import string:

- Imports Python's built-in string module, which provides useful string operations and punctuation characters.

import contractions:

- Imports the contractions library to expand contracted words into their full forms. For example, "can't" becomes "cannot".

from nltk.corpus import stopwords:

- Imports the stopwords list from NLTK, which contains common words such as "the", "is", and "and" that are often removed during text preprocessing.

from nltk.tokenize import word_tokenize:

- Imports the word_tokenize() function, which splits a sentence into individual words or tokens.

from textblob import TextBlob:

- Imports the TextBlob library, which provides features such as spelling correction, sentiment analysis, translation, and noun phrase extraction.

from nltk.stem import PorterStemmer, WordNetLemmatizer:

- **Imports two NLP tools:**

1. PorterStemmer – Used for stemming, which reduces words to their root form by removing suffixes.
2. WordNetLemmatizer – Used for lemmatization, which converts words to their correct dictionary (base) form based on their meaning.

Definition of NLTK Download Statements:

nltk.download('punkt_tab')

- Downloads the Punkt tokenizer data, which is required for tokenizing text into words and sentences using functions like word_tokenize().

nltk.download('stopwords')

- Downloads the Stopwords corpus, which contains commonly used words (such as the, is, and) that are often removed during text preprocessing.

nltk.download('wordnet')

- Downloads the WordNet lexical database, which is required for lemmatization using the WordNetLemmatizer.

nltk.download('omw-1.4')

- Downloads the Open Multilingual WordNet (OMW) 1.4 dataset, which provides additional word information and language support to improve lemmatization accuracy.

Code:

```
[1]: pip install contractions

Collecting contractionsNote: you may need to restart the kernel to use updated packages.
  WARNING: Retrying (Retry(total=4, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NewC
llib3.connection.HTTPSConnection object at 0x000001A5B3876450>: Failed to establish a new connection: [Errno 11001] getadd
4/725241b788963b460ce0118bfd5c505dd3d1bdd020ee740f9f39044ed4a7/contractions-0.1.73-py2.py3-none-any.whl.metadata
WARNING: Retrying (Retry(total=4, connect=None, read=None, redirect=None, status=None)) after connection broken by 'ReadTi
(host='files.pythonhosted.org', port=443): Read timed out. (read timeout=15)")': /packages/e2/0f/6f08dd89e9d71380a369b1f5b
e495/textsearch-0.0.24-py2.py3-none-any.whl

[notice] A new release of pip is available: 24.2 -> 26.1.2
[notice] To update, run: C:\Users\Karthik\AppData\Local\Programs\Python\Python312\python.exe -m pip install --upgrade pip

  Downloading contractions-0.1.73-py2.py3-none-any.whl.metadata (1.2 kB)
Collecting textsearch>=0.0.21 (from contractions)
  Downloading textsearch-0.0.24-py2.py3-none-any.whl.metadata (1.2 kB)
Collecting anyascii (from textsearch>=0.0.21->contractions)
  Downloading anyascii-0.3.3-py3-none-any.whl.metadata (1.6 kB)
Collecting pyahocorasick (from textsearch>=0.0.21->contractions)
  Downloading pyahocorasick-2.3.1-cp312-cp312-win_amd64.whl.metadata (14 kB)
Downloading contractions-0.1.73-py2.py3-none-any.whl (8.7 kB)
Downloading textsearch-0.0.24-py2.py3-none-any.whl (7.6 kB)
Downloading anyascii-0.3.3-py3-none-any.whl (345 kB)
Downloading pyahocorasick-2.3.1-cp312-cp312-win_amd64.whl (35 kB)
Installing collected packages: pyahocorasick, anyascii, textsearch, contractions
Successfully installed anyascii-0.3.3 contractions-0.1.73 pyahocorasick-2.3.1 textsearch-0.0.24
```

```
[2]: pip install textblob

Collecting textblob
  Downloading textblob-0.20.0-py3-none-any.whl.metadata (4.0 kB)
Requirement already satisfied: nltk>=3.9 in c:\users\karthik\appdata\local\programs\python\python312\lib\site-packages (from textblob) (3.9.4)
Requirement already satisfied: click in c:\users\karthik\appdata\local\programs\python\python312\lib\site-packages (from nltk>=3.9->textblob) (8.1.8)
Requirement already satisfied: joblib in c:\users\karthik\appdata\local\programs\python\python312\lib\site-packages (from nltk>=3.9->textblob) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in c:\users\karthik\appdata\local\programs\python\python312\lib\site-packages (from nltk>=3.9->textblob)
(2026.5.9)
Requirement already satisfied: tqdm in c:\users\karthik\appdata\local\programs\python\python312\lib\site-packages (from nltk>=3.9->textblob) (4.67.1)
Requirement already satisfied: colorama in c:\users\karthik\appdata\local\programs\python\python312\lib\site-packages (from click->nltk>=3.9->textblob)
(0.4.6)
Downloading textblob-0.20.0-py3-none-any.whl (624 kB)
   ---------------------------------------- 0.0/625.0 kB ? eta -:--:--
   ---------------------------------------- 0.0/625.0 kB ? eta -:--:--
   ---------------------------------------- 0.0/625.0 kB ? eta -:--:--
   ---------------------------------------- 0.0/625.0 kB ? eta -:--:--
   ---------------------------------------- 0.0/625.0 kB ? eta -:--:--
   ---------------------------------------- 0.0/625.0 kB ? eta -:--:--
   ---------------------------------------- 0.0/625.0 kB ? eta -:--:--
   ---------------------------------------- 0.0/625.0 kB ? eta -:--:--
   --------------- ---------------------- 262.1/625.0 kB ? eta -:--:--
   --------------- ---------------------- 262.1/625.0 kB ? eta -:--:--
   --------------- ---------------------- 262.1/625.0 kB ? eta -:--:--
   --------------- ---------------------- 262.1/625.0 kB ? eta -:--:--
   --------------- ---------------------- 262.1/625.0 kB ? eta -:--:--
   --------------- ---------------------- 262.1/625.0 kB ? eta -:--:--
   --------------- ---------------------- 262.1/625.0 kB ? eta -:--:--
   --------------- ---------------------- 262.1/625.0 kB ? eta -:--:--
   ------------------------------ ------ 524.3/625.0 kB 167.7 kB/s eta 0:00:01
   ------------------------------ ------ 524.3/625.0 kB 167.7 kB/s eta 0:00:01
   ------------------------------ ------ 524.3/625.0 kB 167.7 kB/s eta 0:00:01
   ---------------------------------------- 625.0/625.0 kB 169.5 kB/s eta 0:00:00
Installing collected packages: textblob
Successfully installed textblob-0.20.0
Note: you may need to restart the kernel to use updated packages.
```

```python
import nltk
import string
import contractions
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from textblob import TextBlob
from nltk.stem import PorterStemmer, WordNetLemmatizer
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('wordnet')

# Text Normalization
print("Text Normalization")
text1=input("Enter a Sentence: ")
print("Karthik Vinod, TCS2627033")
print(f"Original text: {text1}")
expanded_text=contractions.fix(text1)
print(f"Expanded text: {expanded_text}")

# Tokenization
print("\nTokenization")
tokens=word_tokenize(expanded_text)
print(f"Tokens: {tokens}")

# Lowercase Conversion
print("\nLowercase Conversion")
lowercase_tokens=[token.lower() for token in tokens]
print(f"Lowercase Tokens: {lowercase_tokens}")

# Punctuation Removal
```

```
print("\nPunctuation Removal")
no_punct_tokens=[token for token in lowercase_tokens if token not in
string.punctuation]
print(f"Punctuation Removal: {no_punct_tokens}")

# Stop Word Remover
print("\nStop Word Remover")
stop_words=set(stopwords.words('english'))
no_stop_tokens=[token for token in no_punct_tokens if token not in stop_words]
print(f"Tokens without stop words: {no_stop_tokens}")

# Text Correction
print("\nText Correction")
correct_tokens=[str(TextBlob(token).correct()) for token in no_stop_tokens]
print(f"Token after spelling correction: {correct_tokens}")

# Stemming
print("\nStemming")
print(f"Original Text: {text1}")
tokens=word_tokenize(text1)
print(f"Tokens: {tokens}")
stemmer=PorterStemmer()
stemmed_tokens=[stemmer.stem(token) for token in tokens]
print(f"Stemmed Tokens: {stemmed_tokens}")

# Lemmatization
print("\nLemmatization")
print(f"Original Text: {text1}")
lemmatizer=WordNetLemmatizer()
print(f"Tokens: {tokens}")
lemmatized_tokens=[lemmatizer.lemmatize(token) for token in tokens]
print(f"Lemmatized Tokens: {lemmatized_tokens}")
```

Output:

Input 1: I can't believe the students are eating delicious sandwiches in the cafeteria!

```
Text Normalization
Enter a Sentence:  I can't believe the students are eating delicious sandwiches in the cafeteria!
Karthik Vinod, TCS2627033
Original text: I can't believe the students are eating delicious sandwiches in the cafeteria!
Expanded text: I cannot believe the students are eating delicious sandwiches in the cafeteria!

Tokenization
Tokens: ['I', 'can', 'not', 'believe', 'the', 'students', 'are', 'eating', 'delicious', 'sandwiches', 'in', 'the', 'cafeteria', '!']

Lowercase Conversion
Lowercase Tokens: ['i', 'can', 'not', 'believe', 'the', 'students', 'are', 'eating', 'delicious', 'sandwiches', 'in', 'the', 'cafeteria', '!']

Punctuation Removal
Punctuation Removal: ['i', 'can', 'not', 'believe', 'the', 'students', 'are', 'eating', 'delicious', 'sandwiches', 'in', 'the', 'cafeteria']

Stop Word Remover
Tokens without stop words: ['believe', 'students', 'eating', 'delicious', 'sandwiches', 'cafeteria']

Text Correction
Token after spelling correction: ['believe', 'students', 'eating', 'delicious', 'sandwiched', 'cafeteria']

Stemming
Original Text: I can't believe the students are eating delicious sandwiches in the cafeteria!
Tokens: ['I', 'ca', "n't", 'believe', 'the', 'students', 'are', 'eating', 'delicious', 'sandwiches', 'in', 'the', 'cafeteria', '!']
Stemmed Tokens: ['i', 'ca', "n't", 'believ', 'the', 'student', 'are', 'eat', 'delici', 'sandwich', 'in', 'the', 'cafeteria', '!']

Lemmatization
Original Text: I can't believe the students are eating delicious sandwiches in the cafeteria!
Tokens: ['I', 'ca', "n't", 'believe', 'the', 'students', 'are', 'eating', 'delicious', 'sandwiches', 'in', 'the', 'cafeteria', '!']
Lemmatized Tokens: ['I', 'ca', "n't", 'believe', 'the', 'student', 'are', 'eating', 'delicious', 'sandwich', 'in', 'the', 'cafeteria', '!']
```

Input 2: She is running quickly towards the lirary, but she forgot her notebok.

```
Text Normalization
Enter a Sentence:  She is running quickly towards the lirary, but she forgot her notebok.
Karthik Vinod, TCS2627033
Original text: She is running quickly towards the lirary, but she forgot her notebok.
Expanded text: She is running quickly towards the lirary, but she forgot her notebok.

Tokenization
Tokens: ['She', 'is', 'running', 'quickly', 'towards', 'the', 'lirary', ',', 'but', 'she', 'forgot', 'her', 'notebok', '.']

Lowercase Conversion
Lowercase Tokens: ['she', 'is', 'running', 'quickly', 'towards', 'the', 'lirary', ',', 'but', 'she', 'forgot', 'her', 'notebok', '.']

Punctuation Removal
Punctuation Removal: ['she', 'is', 'running', 'quickly', 'towards', 'the', 'lirary', 'but', 'she', 'forgot', 'her', 'notebok']

Stop Word Remover
Tokens without stop words: ['running', 'quickly', 'towards', 'lirary', 'forgot', 'notebok']

Text Correction
Token after spelling correction: ['running', 'quickly', 'towards', 'library', 'forgot', 'notebook']

Stemming
Original Text: She is running quickly towards the lirary, but she forgot her notebok.
Tokens: ['She', 'is', 'running', 'quickly', 'towards', 'the', 'lirary', ',', 'but', 'she', 'forgot', 'her', 'notebok', '.']
Stemmed Tokens: ['she', 'is', 'run', 'quickli', 'toward', 'the', 'lirari', ',', 'but', 'she', 'forgot', 'her', 'notebok', '.']

Lemmatization
Original Text: She is running quickly towards the lirary, but she forgot her notebok.
Tokens: ['She', 'is', 'running', 'quickly', 'towards', 'the', 'lirary', ',', 'but', 'she', 'forgot', 'her', 'notebok', '.']
Lemmatized Tokens: ['She', 'is', 'running', 'quickly', 'towards', 'the', 'lirary', ',', 'but', 'she', 'forgot', 'her', 'notebok', '.']
```

Input 3: Ths sentnce has sme speling erors and it won't look correct.

```
Text Normalization
Enter a Sentence:  Ths sentnce has sme speling erors and it won't look correct.
Karthik Vinod, TCS2627033
Original text: Ths sentnce has sme speling erors and it won't look correct.
Expanded text: Ths sentnce has sme speling erors and it will not look correct.

Tokenization
Tokens: ['Ths', 'sentnce', 'has', 'sme', 'speling', 'erors', 'and', 'it', 'will', 'not', 'look', 'correct', '.']

Lowercase Conversion
Lowercase Tokens: ['ths', 'sentnce', 'has', 'sme', 'speling', 'erors', 'and', 'it', 'will', 'not', 'look', 'correct', '.']

Punctuation Removal
Punctuation Removal: ['ths', 'sentnce', 'has', 'sme', 'speling', 'erors', 'and', 'it', 'will', 'not', 'look', 'correct']

Stop Word Remover
Tokens without stop words: ['ths', 'sentnce', 'sme', 'speling', 'erors', 'look', 'correct']

Text Correction
Token after spelling correction: ['the', 'sentence', 'she', 'spelling', 'errors', 'look', 'correct']

Stemming
Original Text: Ths sentnce has sme speling erors and it won't look correct.
Tokens: ['Ths', 'sentnce', 'has', 'sme', 'speling', 'erors', 'and', 'it', 'wo', "n't", 'look', 'correct', '.']
Stemmed Tokens: ['th', 'sentnc', 'ha', 'sme', 'spele', 'eror', 'and', 'it', 'wo', "n't", 'look', 'correct', '.']

Lemmatization
Original Text: Ths sentnce has sme speling erors and it won't look correct.
Tokens: ['Ths', 'sentnce', 'has', 'sme', 'speling', 'erors', 'and', 'it', 'wo', "n't", 'look', 'correct', '.']
Lemmatized Tokens: ['Ths', 'sentnce', 'ha', 'sme', 'speling', 'erors', 'and', 'it', 'wo', "n't", 'look', 'correct', '.']
```

NLP:

Practical No 4:

Name: Karthik Vinod

Roll No: TCS2627033

Aim: Program to perform chunking of noun phrase from a given text: NN - Noun, VBZ - Verbs, JJ - Adjectives, RB - Adverbs, PRP - Pronouns, IN - Prepositions, CC - Conjunctions, DT - Determiners, UH - Interjunction.

Python Libraries & Modules:

import nltk:

- Imports the Natural Language Toolkit (NLTK), the main Python library used for Natural Language Processing (NLP). It provides tools and datasets for tasks such as tokenization, POS tagging, parsing, stemming, lemmatization, and text analysis.

NLTK Data Downloads

nltk.download('averaged_perceptron_tagger_eng'):

- Downloads the pre-trained Averaged Perceptron Tagger for English, which is used to assign Part-of-Speech (POS) tags such as noun (NN), verb (VB), adjective (JJ), etc., to words in a sentence.

nltk.download('punkt_tab'):

- Downloads the Punkt tokenizer dataset, which enables the word_tokenize() function to accurately split text into words and sentences while correctly handling punctuation and abbreviations.

NLTK Processing Tools:

from nltk.tokenize import word_tokenize:

- Imports the word_tokenize() function, which converts a sentence into a list of individual words or tokens. It handles punctuation and special characters more accurately than the basic split() function.

word_tokenize(text):

- Splits the input sentence into individual words (tokens), which serve as the input for further NLP processing.

nltk.pos_tag(words):

- Assigns a Part-of-Speech (POS) tag to each token, identifying its grammatical role such as noun, verb, adjective, determiner, etc.

grammar = "NP: {<DT>?<JJ>*<NN>}":

- Defines a chunk grammar for identifying Noun Phrases (NP). This rule specifies that a noun phrase may contain an optional determiner (<DT>?), zero or more adjectives (<JJ>*), followed by a noun (<NN>).

nltk.RegexpParser(grammar):

- Creates a Regular Expression Parser using the specified grammar. The parser uses the grammar rules to identify and group related words into chunks.

chunk_parser.parse(pos_tags):

- Applies the chunk parser to the POS-tagged words and generates a chunk tree, grouping words that match the grammar into phrases.

tree.pretty_print():

- Displays the generated chunk tree in a well-formatted, easy-to-read tree structure, making it easier to visualize the identified chunks.

Code:

[1]:
```
import nltk
nltk.download("averaged_perceptron_tagger_eng")
nltk.download("punkt_tab")
```

```
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     C:\Users\Karthik\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping taggers\averaged_perceptron_tagger_eng.zip.
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\Karthik\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

```
from nltk.tokenize import word_tokenize
def chunk(text):
    words=word_tokenize(text)
    print(f"Words:{words}")
    pos_tags=nltk.pos_tag(words)
    print(f"POS Tags: {pos_tags}")
```

```
    grammar="NP: {<DT>?<JJ>*<NN>}"
    chunk_parser=nltk.RegexpParser(grammar)
    tree=chunk_parser.parse(pos_tags)
    print(f"Chunk Tree: {tree}")
    tree.pretty_print()

if __name__=="__main__":
    print("Karthik Vinod, TCS2627033")
    user_text=input("Enter a Senetence to Chunk: ")
    chunk(user_text)
```

Output:

Input 1: The quick brown fox jumps over the lazy dog.

```
Karthik Vinod, TCS2627033
Enter a Senetence to Chunk:  The quick brown fox jumps over the lazy dog
Words:['The', 'quick', 'brown', 'fox', 'jumps', 'over', 'the', 'lazy', 'dog']
POS Tags: [('The', 'DT'), ('quick', 'JJ'), ('brown', 'NN'), ('fox', 'NN'), ('jumps', 'VBZ'), ('over', 'IN'), ('the', 'DT'), ('lazy', 'JJ'), ('dog', 'N
N')]
Chunk Tree: (S
  (NP The/DT quick/JJ brown/NN)
  (NP fox/NN)
  jumps/VBZ
  over/IN
  (NP the/DT lazy/JJ dog/NN))
                                  S
    ______________________________|_____________________
   |        |              NP              NP            NP
   |        |       _______|_______        |       ______|_____
jumps/VBZ over/IN The/DT quick/JJ brown/NN fox/NN the/DT lazy/JJ dog/NN
```

Input 2: The brightly colored tropical bird flew across the dense green jungle.

```
Karthik Vinod, TCS2627033
Enter a Senetence to Chunk:  The brightly colored tropical bird flew across the dense green jungle.
Words:['The', 'brightly', 'colored', 'tropical', 'bird', 'flew', 'across', 'the', 'dense', 'green', 'jungle', '.']
POS Tags: [('The', 'DT'), ('brightly', 'RB'), ('colored', 'JJ'), ('tropical', 'JJ'), ('bird', 'NN'), ('flew', 'VBD'), ('across', 'IN'), ('the', 'DT'),
('dense', 'NN'), ('green', 'JJ'), ('jungle', 'NN'), ('.', '.')]
Chunk Tree: (S
  The/DT
  brightly/RB
  (NP colored/JJ tropical/JJ bird/NN)
  flew/VBD
  across/IN
  (NP the/DT dense/NN)
  (NP green/JJ jungle/NN)
  ./.)
                                        S
   _____________________________________|_____________________________________________
  |        |        |        |     |                 NP                  NP            NP
  |        |        |        |     |     ____________|________        ____|____        |
The/DT brightly/RB flew/VBD across/IN ./. colored/JJ tropical/JJ bird/NN the/DT     dense/NN green/JJ     jungle/NN
```

Input 3: The experienced software engineer developed an innovative mobile application.

```
Karthik Vinod, TCS2627033
Enter a Senetence to Chunk:  The experienced software engineer developed an innovative mobile application.
Words:['The', 'experienced', 'software', 'engineer', 'developed', 'an', 'innovative', 'mobile', 'application', '.']
POS Tags: [('The', 'DT'), ('experienced', 'JJ'), ('software', 'NN'), ('engineer', 'NN'), ('developed', 'VBD'), ('an', 'DT'), ('innovative', 'JJ'), ('mobi
le', 'JJ'), ('application', 'NN'), ('.', '.')]
Chunk Tree: (S
  (NP The/DT experienced/JJ software/NN)
  (NP engineer/NN)
  developed/VBD
  (NP an/DT innovative/JJ mobile/JJ application/NN)
  ./.)
                                    S
     _______________________________|_____________________________
    |       |              NP                  NP              NP
    |       |     _________|____________       |       ________|_____________________
developed/VBD ./. The/DT experienced/JJ software/NN engineer/NN an/DT innovative/JJ mobile/JJ application/NN
```

NLP:

Practical No 5:

Name: Karthik Vinod

Roll No: TCS2627033

Aim: Program to create WordCloud from a Wikipedia page.

Python Libraries & Modules:

!pip install wordcloud:

- Installs the WordCloud library, which is used to generate word cloud visualizations from text data by displaying frequently occurring words in larger sizes.

!pip install wikipedia-api:

- Installs the Wikipedia API library, which allows Python programs to access and retrieve information from Wikipedia pages.

import matplotlib.pyplot as plt:

- Imports the Matplotlib plotting library, which is used to create and display graphical visualizations such as the generated word cloud.

from wordcloud import WordCloud, STOPWORDS:

- Imports the WordCloud class for creating word cloud visualizations and the STOPWORDS collection, which contains common words (such as the, is, and and) that are ignored when generating the word cloud.

import wikipediaapi:

- Imports the Wikipedia API library, which provides an interface to retrieve information from Wikipedia.

Wikipedia API Initialization:

wiki = wikipediaapi.Wikipedia(user_agent='Colab', language='en'):

- Initializes the Wikipedia API by specifying the user agent (which identifies the application making the request) and setting the language to English.

page_py = wiki.page("Cristiano Ronaldo"):

- Retrieves the Wikipedia page object for Cristiano Ronaldo.

text = page_py.text:

- Extracts the complete text content from the retrieved Wikipedia page. This text serves as the input for generating the word cloud.

Code:

```
[11]: !pip install wordcloud

Requirement already satisfied: wordcloud in c:\users\karthik\appdata\local\programs\python\python313\lib\site-packages (1.9.6)
Requirement already satisfied: numpy>=1.19.3 in c:\users\karthik\appdata\local\programs\python\python313\lib\site-packages (from wordcloud) (2.5.1)
Requirement already satisfied: pillow in c:\users\karthik\appdata\local\programs\python\python313\lib\site-packages (from wordcloud) (12.3.0)
Requirement already satisfied: matplotlib in c:\users\karthik\appdata\local\programs\python\python313\lib\site-packages (from wordcloud) (3.11.1)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\karthik\appdata\local\programs\python\python313\lib\site-packages (from matplotlib->wordclou
d) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\users\karthik\appdata\local\programs\python\python313\lib\site-packages (from matplotlib->wordcloud)
(0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in c:\users\karthik\appdata\local\programs\python\python313\lib\site-packages (from matplotlib->wordclou
d) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\karthik\appdata\local\programs\python\python313\lib\site-packages (from matplotlib->wordclou
d) (1.5.0)
Requirement already satisfied: packaging>=20.0 in c:\users\karthik\appdata\local\programs\python\python313\lib\site-packages (from matplotlib->wordcloud)
(26.2)
Requirement already satisfied: pyparsing>=3 in c:\users\karthik\appdata\local\programs\python\python313\lib\site-packages (from matplotlib->wordcloud)
(3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\karthik\appdata\local\programs\python\python313\lib\site-packages (from matplotlib->wordc
loud) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\karthik\appdata\local\programs\python\python313\lib\site-packages (from python-dateutil>=2.7->matplot
lib->wordcloud) (1.17.0)

[notice] A new release of pip is available: 25.3 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
[12]: !pip install wikipedia-api

Requirement already satisfied: wikipedia-api in c:\users\karthik\appdata\local\programs\python\python313\lib\site-packages (0.15.0)
Requirement already satisfied: click==8.4.1 in c:\users\karthik\appdata\local\programs\python\python313\lib\site-packages (from wikipedia-api) (8.4.1)
Requirement already satisfied: httpx==0.28.1 in c:\users\karthik\appdata\local\programs\python\python313\lib\site-packages (from wikipedia-api) (0.28.1)
Requirement already satisfied: tenacity==9.1.4 in c:\users\karthik\appdata\local\programs\python\python313\lib\site-packages (from wikipedia-api) (9.1.4)
Requirement already satisfied: colorama in c:\users\karthik\appdata\local\programs\python\python313\lib\site-packages (from click==8.4.1->wikipedia-api)
(0.4.6)
Requirement already satisfied: anyio in c:\users\karthik\appdata\local\programs\python\python313\lib\site-packages (from httpx==0.28.1->wikipedia-api)
(4.14.2)
Requirement already satisfied: certifi in c:\users\karthik\appdata\local\programs\python\python313\lib\site-packages (from httpx==0.28.1->wikipedia-api)
(2025.11.12)
Requirement already satisfied: httpcore==1.* in c:\users\karthik\appdata\local\programs\python\python313\lib\site-packages (from httpx==0.28.1->wikipedia
-api) (1.0.9)
Requirement already satisfied: idna in c:\users\karthik\appdata\local\programs\python\python313\lib\site-packages (from httpx==0.28.1->wikipedia-api) (3.
11)
Requirement already satisfied: h11>=0.16 in c:\users\karthik\appdata\local\programs\python\python313\lib\site-packages (from httpcore==1.*->httpx==0.28.1
->wikipedia-api) (0.16.0)

[notice] A new release of pip is available: 25.3 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
import matplotlib.pyplot as plt
from wordcloud import WordCloud, STOPWORDS
import wikipediaapi

wiki=wikipediaapi.Wikipedia(user_agent='Colab', language='en')
page_py=wiki.page("Cristiano Ronaldo")
text=page_py.text
print("Karthik Vinod, TCS2627033")
print(text)

def plot_wordcloud(wc):
    plt.axis("off")
    plt.figure(figsize=(7,7))
```

```
    plt.imshow(wc)
    plt.show()
wc=WordCloud(width=500, height=500, background_color="red",
stopwords=STOPWORDS, random_state=1).generate(text)
print("Karthik Vinod, TCS2627033")
plot_wordcloud(wc)
```

Output:

```
Karthik Vinod, TCS2627033
Cristiano Ronaldo dos Santos Aveiro (born 5 February 1985) is a Portuguese professional footballer who plays as a forward for and captains the Saudi Pr
o League club Al-Nassr and the Portugal national team. Widely regarded as one of the greatest players in history, he has won numerous individual accola
des throughout his career, including five Ballon d'Or awards, a record three UEFA Men's Player of the Year Awards, and four European Golden Shoes. He w
as named the world's best player five times by FIFA.
Nicknamed CR7, Ronaldo is one of the most decorated players in the history of professional football, having won 35 trophies in his career, including fi
ve UEFA Champions Leagues, two UEFA Nations Leagues and the UEFA European Championship. He holds the records for most goals (140) and assists (42) in t
he Champions League, most goals (14) and assists (8) in the European Championship, most international appearances (233), most men's international goals
(146), and most international victories (141). He is the only player to have scored 100 goals with four different clubs and to finish as top scorer in
four different domestic leagues. He has made over 1,300 professional career appearances, the most by an outfield player, and has scored over 970 offici
al senior career goals for club and country, making him the top goalscorer of all time.
Born in Funchal, Madeira, Ronaldo began his career with Sporting CP before signing with Manchester United in 2003. He gradually established himself as
an integral player for the club, winning three consecutive Premier League titles, the 2007–08 Champions League, and the 2008 FIFA Club World Cup. For h
is performances in 2007–08, Ronaldo was awarded his first Ballon d'Or at age 23. In 2009, he joined Real Madrid in a deal worth €94 million (£80 millio
n), which was the most expensive transfer in history at the time. He was at the forefront of Madrid's resurgence as a dominant European force, helping
them win four Champions Leagues between 2014 and 2018, including the long-awaited La Décima in 2014, where he set the record for most goals scored in a
Champions League season. With Madrid, he also won two La Liga titles, won the Ballon d'Or in 2013, 2014, 2016, and 2017, and became the club's all-time
top goalscorer.
```



NLP:

Practical No 6:

Name: Karthik Vinod

Roll No: TCS2627033

Aim: Write a python program to generate a N-Gram model.

Python Libraries & Modules:

- **import nltk:** Imports the Natural Language Toolkit (NLTK), a Python library used for Natural Language Processing (NLP) tasks such as tokenization, text processing, and language analysis.
- **from nltk.corpus import stopwords:** Imports the stopwords corpus from NLTK, which contains commonly used words (such as the, is, and and) that are often removed before text analysis.

NLTK Data Downloads

- **nltk.download('stopwords'):** Downloads the Stopwords corpus, which provides a list of common words that can be removed during preprocessing.
- **nltk.download('punkt_tab'):** Downloads the Punkt tokenizer dataset, which is required by the word_tokenize() function to accurately split text into words and sentences.

Code:

```
[2]: import nltk
     from nltk.corpus import stopwords
     nltk.download('stopwords')
     nltk.download('punkt_tab')

     [nltk_data] Downloading package stopwords to
     [nltk_data]     C:\Users\Karthik\AppData\Roaming\nltk_data...
     [nltk_data]   Package stopwords is already up-to-date!
     [nltk_data] Downloading package punkt_tab to
     [nltk_data]     C:\Users\Karthik\AppData\Roaming\nltk_data...
     [nltk_data]   Package punkt_tab is already up-to-date!
[2]: True
```

```
import nltk
from nltk.corpus import stopwords
nltk.download('stopwords')
nltk.download('punkt_tab')
```

```
def generateNGram(text, n=1):
    words=nltk.word_tokenize(text)
    words=[word for word in words if word.lower() not in
set(stopwords.words('english'))]
    temp=zip(*[words[i:] for i in range(0,n)])
    ngram=[' '.join(ngram) for ngram in temp]
    return ngram

print("Karthik Vinod, TCS2627033")
txt=input("Enter a sentence to perform N-Gram on it: ")
num=int(input("Enter the value of 'n' in N-Gram Model: "))
generateNGram(txt,num)
```

Output:

Input 1: My name is Karthik and my Roll Number is TCS2627033

```
Karthik Vinod, TCS2627033
Enter a sentence to perform N-Gram on it:  My name is Karthik and my Roll Number is TCS2627033
Enter the value of 'n' in N-Gram Model:  2
['name Karthik', 'Karthik Roll', 'Roll Number', 'Number TCS2627033']
```

Input 2: Natural language processing helps computers understand human language

```
Karthik Vinod, TCS2627033
Enter a sentence to perform N-Gram on it:  Natural language processing helps computers understand human language
Enter the value of 'n' in N-Gram Model:  3
['Natural language processing',
 'language processing helps',
 'processing helps computers',
 'helps computers understand',
 'computers understand human',
 'understand human language']
```

Input 3: Python is a popular programming language for data science and artificial intelligence

```
Karthik Vinod, TCS2627033
Enter a sentence to perform N-Gram on it:  Python is a popular programming language for data science and artificial intelligence
Enter the value of 'n' in N-Gram Model:  4
['Python popular programming language',
 'popular programming language data',
 'programming language data science',
 'language data science artificial',
 'data science artificial intelligence']
```

NLP:

Practical No 7:

Name: Karthik Vinod

Roll No: TCS2627033

Aim: Write a program to study the Brown Corpus.

Brown Corpus – Description:

- The Brown Corpus is one of the first and most widely used electronic collections of English text for Natural Language Processing (NLP).
- It was created in 1961 at Brown University by Henry Kučera and W. Nelson Francis.
- The corpus contains about one million words collected from 500 different English documents published in the United States.
- These documents cover a wide range of genres, including news articles, fiction, government reports, editorials, academic writing, biographies, and other forms of text.

The Brown Corpus is commonly used to:

- Study word frequency and language patterns.
- Train and evaluate NLP and machine learning models.
- Perform Part-of-Speech (POS) tagging.
- Generate N-Grams and statistical language models.
- Analyze vocabulary and writing styles across different text categories.

Python Libraries & Modules:

- **import nltk:** Imports the Natural Language Toolkit (NLTK), a Python library used for Natural Language Processing (NLP) tasks such as tokenization, corpus analysis, POS tagging, and text processing.

NLTK Data Downloads:

- **nltk.download('brown'):** Downloads the Brown Corpus, one of the most widely used text corpora in NLP. It contains approximately 1 million words collected from 500 English documents across different categories such as news, fiction, editorials,

government reports, and reviews. The corpus is commonly used for language modeling, text analysis, and machine learning applications.

Brown Corpus Module:

- **from nltk.corpus import brown:** Imports the Brown Corpus from NLTK, allowing the program to access its collection of English text. The corpus can be used to retrieve words, sentences, categories, and documents for various NLP tasks such as word frequency analysis, N-Gram generation, and language modeling.

Code & Output:

```
import nltk
nltk.download('brown')
from nltk.corpus import brown

# Check the Availability of the File ID in the Corpus
print(f"File ID's of Brown Corpus is {brown.fileids()}")
```

```
File ID's of Brown Corpus is ['ca01', 'ca02', 'ca03', 'ca04', 'ca05', 'ca06', 'ca07', 'ca08', 'ca09', 'ca10', 'ca11', 'ca12', 'ca13', 'ca14', 'ca15', 'ca
16', 'ca17', 'ca18', 'ca19', 'ca20', 'ca21', 'ca22', 'ca23', 'ca24', 'ca25', 'ca26', 'ca27', 'ca28', 'ca29', 'ca30', 'ca31', 'ca32', 'ca33', 'ca34', 'ca
5', 'ca36', 'ca37', 'ca38', 'ca39', 'ca40', 'ca41', 'ca42', 'ca43', 'ca44', 'cb01', 'cb02', 'cb03', 'cb04', 'cb05', 'cb06', 'cb07', 'cb08', 'cb09', 'cb1
0', 'cb11', 'cb12', 'cb13', 'cb14', 'cb15', 'cb16', 'cb17', 'cb18', 'cb19', 'cb20', 'cb21', 'cb22', 'cb23', 'cb24', 'cb25', 'cb26', 'cb27', 'cc01', 'cc0
2', 'cc03', 'cc04', 'cc05', 'cc06', 'cc07', 'cc08', 'cc09', 'cc10', 'cc11', 'cc12', 'cc13', 'cc14', 'cc15', 'cc16', 'cc17', 'cd01', 'cd02', 'cd03', 'cd0
4', 'cd05', 'cd06', 'cd07', 'cd08', 'cd09', 'cd10', 'cd11', 'cd12', 'cd13', 'cd14', 'cd15', 'cd16', 'cd17', 'ce01', 'ce02', 'ce03', 'ce04', 'ce05', 'ce0
6', 'ce07', 'ce08', 'ce09', 'ce10', 'ce11', 'ce12', 'ce13', 'ce14', 'ce15', 'ce16', 'ce17', 'ce18', 'ce19', 'ce20', 'ce21', 'ce22', 'ce23', 'ce24', 'ce2
5', 'ce26', 'ce27', 'ce28', 'ce29', 'ce30', 'ce31', 'ce32', 'ce33', 'ce34', 'ce35', 'ce36', 'cf01', 'cf02', 'cf03', 'cf04', 'cf05', 'cf06', 'cf07', 'cf0
8', 'cf09', 'cf10', 'cf11', 'cf12', 'cf13', 'cf14', 'cf15', 'cf16', 'cf17', 'cf18', 'cf19', 'cf20', 'cf21', 'cf22', 'cf23', 'cf24', 'cf25', 'cf26', 'cf2
7', 'cf28', 'cf29', 'cf30', 'cf31', 'cf32', 'cf33', 'cf34', 'cf35', 'cf36', 'cf37', 'cf38', 'cf39', 'cf40', 'cf41', 'cf42', 'cf43', 'cf44', 'cf45', 'cf4
6', 'cf47', 'cf48', 'cg01', 'cg02', 'cg03', 'cg04', 'cg05', 'cg06', 'cg07', 'cg08', 'cg09', 'cg10', 'cg11', 'cg12', 'cg13', 'cg14', 'cg15', 'cg16', 'cg1
7', 'cg18', 'cg19', 'cg20', 'cg21', 'cg22', 'cg23', 'cg24', 'cg25', 'cg26', 'cg27', 'cg28', 'cg29', 'cg30', 'cg31', 'cg32', 'cg33', 'cg34', 'cg35', 'cg3
6', 'cg37', 'cg38', 'cg39', 'cg40', 'cg41', 'cg42', 'cg43', 'cg44', 'cg45', 'cg46', 'cg47', 'cg48', 'cg49', 'cg50', 'cg51', 'cg52', 'cg53', 'cg54', 'cg5
5', 'cg56', 'cg57', 'cg58', 'cg59', 'cg60', 'cg61', 'cg62', 'cg63', 'cg64', 'cg65', 'cg66', 'cg67', 'cg68', 'cg69', 'cg70', 'cg71', 'cg72', 'cg73', 'cg7
4', 'cg75', 'ch01', 'ch02', 'ch03', 'ch04', 'ch05', 'ch06', 'ch07', 'ch08', 'ch09', 'ch10', 'ch11', 'ch12', 'ch13', 'ch14', 'ch15', 'ch16', 'ch17', 'ch1
8', 'ch19', 'ch20', 'ch21', 'ch22', 'ch23', 'ch24', 'ch25', 'ch26', 'ch27', 'ch28', 'ch29', 'ch30', 'cj01', 'cj02', 'cj03', 'cj04', 'cj05', 'cj06', 'cj0
7', 'cj08', 'cj09', 'cj10', 'cj11', 'cj12', 'cj13', 'cj14', 'cj15', 'cj16', 'cj17', 'cj18', 'cj19', 'cj20', 'cj21', 'cj22', 'cj23', 'cj24', 'cj25', 'cj2
6', 'cj27', 'cj28', 'cj29', 'cj30', 'cj31', 'cj32', 'cj33', 'cj34', 'cj35', 'cj36', 'cj37', 'cj38', 'cj39', 'cj40', 'cj41', 'cj42', 'cj43', 'cj44', 'cj4
5', 'cj46', 'cj47', 'cj48', 'cj49', 'cj50', 'cj51', 'cj52', 'cj53', 'cj54', 'cj55', 'cj56', 'cj57', 'cj58', 'cj59', 'cj60', 'cj61', 'cj62', 'cj63', 'cj6
4', 'cj65', 'cj66', 'cj67', 'cj68', 'cj69', 'cj70', 'cj71', 'cj72', 'cj73', 'cj74', 'cj75', 'cj76', 'cj77', 'cj78', 'cj79', 'cj80', 'ck01', 'ck02', 'ck0
3', 'ck04', 'ck05', 'ck06', 'ck07', 'ck08', 'ck09', 'ck10', 'ck11', 'ck12', 'ck13', 'ck14', 'ck15', 'ck16', 'ck17', 'ck18', 'ck19', 'ck20', 'ck21', 'ck2
2', 'ck23', 'ck24', 'ck25', 'ck26', 'ck27', 'ck28', 'ck29', 'cl01', 'cl02', 'cl03', 'cl04', 'cl05', 'cl06', 'cl07', 'cl08', 'cl09', 'cl10', 'cl11', 'cl1
2', 'cl13', 'cl14', 'cl15', 'cl16', 'cl17', 'cl18', 'cl19', 'cl20', 'cl21', 'cl22', 'cl23', 'cl24', 'cm01', 'cm02', 'cm03', 'cm04', 'cm05', 'cm06', 'cn0
1', 'cn02', 'cn03', 'cn04', 'cn05', 'cn06', 'cn07', 'cn08', 'cn09', 'cn10', 'cn11', 'cn12', 'cn13', 'cn14', 'cn15', 'cn16', 'cn17', 'cn18', 'cn19', 'cn2
0', 'cn21', 'cn22', 'cn23', 'cn24', 'cn25', 'cn26', 'cn27', 'cn28', 'cn29', 'cp01', 'cp02', 'cp03', 'cp04', 'cp05', 'cp06', 'cp07', 'cp08', 'cp09', 'cp1
0', 'cp11', 'cp12', 'cp13', 'cp14', 'cp15', 'cp16', 'cp17', 'cp18', 'cp19', 'cp20', 'cp21', 'cp22', 'cp23', 'cp24', 'cp25', 'cp26', 'cp27', 'cp28', 'cp2
9', 'cr01', 'cr02', 'cr03', 'cr04', 'cr05', 'cr06', 'cr07', 'cr08', 'cr09']
```

```
# Exploring file ID ca01, the no. of words in it and also to get the words
from it
ca01=brown.words('ca01')
print(f'Words in ca01 are as follows: {ca01}')
```

```
Words in ca01 are as follows: ['The', 'Fulton', 'County', 'Grand', 'Jury', 'said', ...]
```

```
# To Check the lenght of the words associated with the appropriate file ID
print(f'ca01 has {len(ca01)} Words')
```

```
ca01 has 2242 Words
```

```
# Exploring the no. of categories present in the brown corpus
print(f'Categories in Brown Corpus are: \n{brown.categories()}')
```

```
Categories in Brown Corpus are:
['adventure', 'belles_lettres', 'editorial', 'fiction', 'government', 'hobbies', 'humor', 'learned', 'lore', 'mystery', 'news', 'religion', 'reviews', 'r
omance', 'science_fiction']
```

```
# Statistics of Various Words
print("Karthik Vinod, TCS2627033")
print("Average Word Length\t Average Sentence Length\t\t Number of Time each
word appears on Average\t\t File Name")
for fileid in brown.fileids():
    num_chars=len(brown.raw(fileid))
    num_words=len(brown.words(fileid))
    num_sent=len(brown.sents(fileid))
    num_vocab=len(set([w.lower() for w in brown.raw(fileid)]))
    print(f'{int(num_chars/num_words)}\t\t\t\t
{int(num_words/num_sent)}\t\t\t\t {int(num_words/num_vocab)}\t\t\t\t\t\t\t
{fileid}')
```

```
Karthik Vinod, TCS2627033
Average Word Length      Average Sentence Length                Number of Time each word appears on Average             File Name
9                                22                                      44                                              ca01
8                                23                                      43                                              ca02
8                                20                                      43                                              ca03
9                                25                                      44                                              ca04
8                                26                                      47                                              ca05
8                                22                                      46                                              ca06
9                                18                                      43                                              ca07
8                                21                                      43                                              ca08
9                                19                                      42                                              ca09
8                                21                                      44                                              ca10
8                                22                                      43                                              ca11
8                                22                                      43                                              ca12
8                                20                                      43                                              ca13
8                                17                                      43                                              ca14
8                                21                                      43                                              ca15
8                                20                                      43                                              ca16
8                                22                                      43                                              ca17
8                                22                                      46                                              ca18
8                                20                                      42                                              ca19
8                                23                                      46                                              ca20
9                                20                                      43                                              ca21
8                                21                                      47                                              ca22
8                                20                                      43                                              ca23
8                                22                                      44                                              ca24
8                                21                                      44                                              ca25
8                                21                                      42                                              ca26
8                                19                                      41                                              ca27
8                                23                                      41                                              ca28
8                                19                                      43                                              ca29
8                                19                                      46                                              ca30
8                                20                                      43                                              ca31
8                                18                                      43                                              ca32
```

NLP:

Practical No 8:

Name: Karthik Vinod

Roll No: TCS2627033

Aim: Write a python program to perform frequency based extractive summarization.

Python Libraries & Modules:

- **import nltk:** Imports the Natural Language Toolkit (NLTK), a Python library used for Natural Language Processing (NLP) tasks such as tokenization, stop word removal, text summarization, and language analysis.
- **from nltk.corpus import stopwords:** Imports the stopwords corpus, which contains commonly used English words (such as the, is, and and) that are removed during text preprocessing.
- **from nltk.tokenize import word_tokenize, sent_tokenize:** Imports the word_tokenize() function to split text into words and the sent_tokenize() function to split text into individual sentences.

NLTK Data Downloads:

- **nltk.download('stopwords'):** Downloads the Stopwords corpus, which provides a list of common English words that are removed before text analysis.
- **nltk.download('punkt_tab'):** Downloads the Punkt tokenizer dataset, which is required for accurately splitting text into words and sentences.

Text Summarization Processing:

- **word_tokenize(text):** Splits the input text into individual words (tokens).
- **sent_tokenize(text):** Splits the input paragraph into individual sentences.
- **wordfreq = dict():** Creates an empty dictionary to store the frequency of each word.
- **sentrank = dict():** Creates an empty dictionary to store the score (rank) of each sentence based on word frequencies.
- **summary = "":** Creates an empty string to store the generated summary.
- **summary = summary + sentence:** Adds high-ranking sentences to the final summary.
- **print(f"Summary: {summary}"):** Displays the summarized text generated by the program.

Code & Output:

```
[1]: import nltk
     from nltk.corpus import stopwords
     from nltk.tokenize import word_tokenize, sent_tokenize
     nltk.download('stopwords')
     nltk.download('punkt_tab')

     [nltk_data] Downloading package stopwords to
     [nltk_data]     C:\Users\admin\AppData\Roaming\nltk_data...
     [nltk_data]   Package stopwords is already up-to-date!
     [nltk_data] Downloading package punkt_tab to
     [nltk_data]     C:\Users\admin\AppData\Roaming\nltk_data...
     [nltk_data]   Package punkt_tab is already up-to-date!
[1]: True
```

```
text= """NLTK is a leading platform for building Python programs to work with
human language data. It provides easy-to-use interfaces to over 50 corpora and
lexical resources such as WordNet, along with a suite of text processing
libraries for classification, tokenization, stemming, tagging, parsing, and
semantic reasoning, wrappers for industrial-strength NLP libraries, and an
active discussion forum.Thanks to a hands-on guide introducing programming
fundamentals alongside topics in computational linguistics, plus comprehensive
API documentation, NLTK is suitable for linguists, engineers, students,
educators, researchers, and industry users alike. NLTK is available for
Windows, macOS, and Linux. Best of all, NLTK is a free, open source,
community-driven project.
NLTK has been called “a wonderful tool for teaching, and working in,
computational linguistics using Python,” and “an amazing library to play with
natural language.Natural Language Processing with Python provides a practical
introduction to programming for language processing. Written by the creators
of NLTK, it guides the reader through the fundamentals of writing Python
programs, working with corpora, categorizing text, analyzing linguistic
structure, and more. The online version of the book has been been updated for
Python 3 and NLTK 3"""
print("Karthik Vinod, TCS2627033")
print(text)
```

```
Karthik Vinod, TCS2627033
NLTK is a leading platform for building Python programs to work with human language data. It provides easy-to-use interfaces to over 50 corpora and lex
ical resources such as WordNet, along with a suite of text processing libraries for classification, tokenization, stemming, tagging, parsing, and seman
tic reasoning, wrappers for industrial-strength NLP libraries, and an active discussion forum.Thanks to a hands-on guide introducing programming fundam
entals alongside topics in computational linguistics, plus comprehensive API documentation, NLTK is suitable for linguists, engineers, students, educat
ors, researchers, and industry users alike. NLTK is available for Windows, macOS, and Linux. Best of all, NLTK is a free, open source, community-driven
project.
NLTK has been called “a wonderful tool for teaching, and working in, computational linguistics using Python,” and “an amazing library to play with natu
ral language.Natural Language Processing with Python provides a practical introduction to programming for language processing. Written by the creators
of NLTK, it guides the reader through the fundamentals of writing Python programs, working with corpora, categorizing text, analyzing linguistic struct
ure, and more. The online version of the book has been been updated for Python 3 and NLTK 3
```

```
words=nltk.word_tokenize(text)
words=[word for word in words if word.lower() not in
set(stopwords.words('english'))]
```

```
print("Karthik Vinod, TCS2627033")
print(words)
```

```
Karthik Vinod, TCS2627033
['NLTK', 'leading', 'platform', 'building', 'Python', 'programs', 'work', 'human', 'language', 'data', '.', 'provides', 'easy-to-use', 'interfaces', '5
0', 'corpora', 'lexical', 'resources', 'WordNet', ',', 'along', 'suite', 'text', 'processing', 'libraries', 'classification', ',', 'tokenization', ',',
'stemming', ',', 'tagging', ',', 'parsing', ',', 'semantic', 'reasoning', ',', 'wrappers', 'industrial-strength', 'NLP', 'libraries', ',', 'active', 'd
iscussion', 'forum.Thanks', 'hands-on', 'guide', 'introducing', 'programming', 'fundamentals', 'alongside', 'topics', 'computational', 'linguistics',
',', 'plus', 'comprehensive', 'API', 'documentation', ',', 'NLTK', 'suitable', 'linguists', ',', 'engineers', ',', 'students', ',', 'educators', ',',
'researchers', ',', 'industry', 'users', 'alike', '.', 'NLTK', 'available', 'Windows', ',', 'macOS', ',', 'Linux', '.', 'Best', ',', 'NLTK', 'free',
',', 'open', 'source', ',', 'community-driven', 'project', '.', 'NLTK', 'called', '“', 'wonderful', 'tool', 'teaching', ',', 'working', ',', 'computati
onal', 'linguistics', 'using', 'Python', ',', '”', '“', 'amazing', 'library', 'play', 'natural', 'language.Natural', 'Language', 'Processing', 'Pytho
n', 'provides', 'practical', 'introduction', 'programming', 'language', 'processing', '.', 'Written', 'creators', 'NLTK', ',', 'guides', 'reader', 'fun
damentals', 'writing', 'Python', 'programs', ',', 'working', 'corpora', ',', 'categorizing', 'text', ',', 'analyzing', 'linguistic', 'structure', ',',
'.', 'online', 'version', 'book', 'updated', 'Python', '3', 'NLTK', '3']
```

```
# Calculate the Frequency of Words
print("Karthik Vinod, TCS2627033")
wordfreq=dict()
for word in words:
    word=word.lower()
    if word in wordfreq:
        wordfreq[word]+=1
    else:
        wordfreq[word]=1
print(wordfreq)
```

```
Karthik Vinod, TCS2627033
{'nltk': 7, 'leading': 1, 'platform': 1, 'building': 1, 'python': 5, 'programs': 2, 'work': 1, 'human': 1, 'language': 3, 'data': 1, '.': 6, 'provide
s': 2, 'easy-to-use': 1, 'interfaces': 1, '50': 1, 'corpora': 2, 'lexical': 1, 'resources': 1, 'wordnet': 1, ',': 28, 'along': 1, 'suite': 1, 'text':
2, 'processing': 3, 'libraries': 2, 'classification': 1, 'tokenization': 1, 'stemming': 1, 'tagging': 1, 'parsing': 1, 'semantic': 1, 'reasoning': 1,
'wrappers': 1, 'industrial-strength': 1, 'nlp': 1, 'active': 1, 'discussion': 1, 'forum.thanks': 1, 'hands-on': 1, 'guide': 1, 'introducing': 1, 'progr
amming': 2, 'fundamentals': 2, 'alongside': 1, 'topics': 1, 'computational': 2, 'linguistics': 2, 'plus': 1, 'comprehensive': 1, 'api': 1, 'documentati
on': 1, 'suitable': 1, 'linguists': 1, 'engineers': 1, 'students': 1, 'educators': 1, 'researchers': 1, 'industry': 1, 'users': 1, 'alike': 1, 'availab
le': 1, 'windows': 1, 'macos': 1, 'linux': 1, 'best': 1, 'free': 1, 'open': 1, 'source': 1, 'community-driven': 1, 'project': 1, 'called': 1, '“': 2,
'wonderful': 1, 'tool': 1, 'teaching': 1, 'working': 2, 'using': 1, '”': 1, 'amazing': 1, 'library': 1, 'play': 1, 'natural': 1, 'language.natural': 1,
'practical': 1, 'introduction': 1, 'written': 1, 'creators': 1, 'guides': 1, 'reader': 1, 'writing': 1, 'categorizing': 1, 'analyzing': 1, 'linguisti
c': 1, 'structure': 1, 'online': 1, 'version': 1, 'book': 1, 'updated': 1, '3': 2}
```

```
# Calculate the rank of a Sentence
print("Karthik Vinod, TCS2627033")
sentences=sent_tokenize(text)
sentrank=dict()
for sentence in sentences:
    for word,freq in wordfreq.items():
        if word in sentence.lower():
            if sentence in sentrank:
                sentrank[sentence]+=freq
            else:
                sentrank[sentence]=freq
print(sentrank)
```

```
Karthik Vinod, TCS2627033
{'NLTK is a leading platform for building Python programs to work with human language data.': 29, 'It provides easy-to-use interfaces to over 50 corpor
a and lexical resources such as WordNet, along with a suite of text processing libraries for classification, tokenization, stemming, tagging, parsing,
and semantic reasoning, wrappers for industrial-strength NLP libraries, and an active discussion forum.Thanks to a hands-on guide introducing programmi
ng fundamentals alongside topics in computational linguistics, plus comprehensive API documentation, NLTK is suitable for linguists, engineers, student
s, educators, researchers, and industry users alike.': 101, 'NLTK is available for Windows, macOS, and Linux.': 45, 'Best of all, NLTK is a free, open
source, community-driven project.': 47, 'NLTK has been called “a wonderful tool for teaching, and working in, computational linguistics using Python,”
and “an amazing library to play with natural language.Natural Language Processing with Python provides a practical introduction to programming for lang
uage processing.': 79, 'Written by the creators of NLTK, it guides the reader through the fundamentals of writing Python programs, working with corpor
a, categorizing text, analyzing linguistic structure, and more.': 67, 'The online version of the book has been been updated for Python 3 and NLTK 3': 1
8}
```

```python
# Summarize the Extract according to Sentence rank
print("Karthik Vinod, TCS2627033")
sumrank=sum(sentrank.values())
averagerank=sumrank/len(sentrank)

summary=""

for sentence in sentences:
    if sentence in sentrank and sentrank[sentence]>1.2*averagerank:
        summary=summary+sentences
print(f"Summary: {summary}")
```

```
Karthik Vinod, TCS2627033
Summary: It provides easy-to-use interfaces to over 50 corpora and lexical resources such as WordNet, along with a suite of text processing libraries f
or classification, tokenization, stemming, tagging, parsing, and semantic reasoning, wrappers for industrial-strength NLP libraries, and an active disc
ussion forum.Thanks to a hands-on guide introducing programming fundamentals alongside topics in computational linguistics, plus comprehensive API docu
mentation, NLTK is suitable for linguists, engineers, students, educators, researchers, and industry users alike.NLTK is available for Windows, macOS,
and Linux.Best of all, NLTK is a free, open source, community-driven project.NLTK has been called “a wonderful tool for teaching, and working in, compu
tational linguistics using Python,” and “an amazing library to play with natural language.Natural Language Processing with Python provides a practical
introduction to programming for language processing.Written by the creators of NLTK, it guides the reader through the fundamentals of writing Python pr
ograms, working with corpora, categorizing text, analyzing linguistic structure, and more.
```

NLP:

Practical No 9:

Name: Karthik Vinod

Roll No: TCS2627033

Aim: Python program to implement syntactic parsing of a given text using Context Free Grammar.

Python Libraries & Modules:

- import nltk: Imports the Natural Language Toolkit (NLTK), a Python library used for Natural Language Processing (NLP) tasks such as tokenization, parsing, grammar analysis, and text processing.
- from nltk.tokenize import word_tokenize: Imports the word_tokenize() function, which splits a sentence into individual words (tokens).

NLTK Data Downloads:

- **nltk.download('punkt_tab'):** Downloads the Punkt tokenizer dataset, which is required by the word_tokenize() function to accurately split text into words and sentences.

Context-Free Grammar (CFG) Parsing:

- **word_tokenize(sentence):** Splits the input sentence into individual words (tokens) before parsing.
- **grammar = """..."""":** Defines the Context-Free Grammar (CFG) rules that describe the valid sentence structure. These rules specify how words and phrases can be combined to form grammatically correct sentences.
- **nltk.CFG.fromstring(grammar):** Converts the grammar rules written as a string into a CFG object that can be used by the parser.
- **parser = nltk.ChartParser(cfg):** Creates a Chart Parser, which uses the defined CFG rules to analyze the sentence and generate its parse tree.
- **parser.parse(words):** Parses the tokenized sentence according to the CFG rules and generates one or more parse trees if the sentence is grammatically valid.
- **print(tree):** Displays the generated parse tree in text format.
- **tree.pretty_print():** Displays the parse tree in a well-formatted, graphical tree structure that is easier to understand.

Code & Output:

```
import nltk
from nltk.tokenize import word_tokenize
nltk.download('punkt_tab')

print("Karthik Vinod, TCS2627033")

words=word_tokenize(text)
grammar="""
S -> VP
VP -> VP NP
NP -> Det NP
Det -> 'that'
NP -> 'flight'
VP -> 'book'
"""

sentence="book that flight"
cfg=nltk.CFG.fromstring(grammar)
words=word_tokenize(sentence)
parser=nltk.ChartParser(cfg)
print("Parse Tree:")
for tree in parser.parse(words):
    print(tree)
tree.pretty_print()
```

```
Karthik Vinod, TCS2627033
Parse Tree:
(S (VP (VP book) (NP (Det that) (NP flight))))
      S
      |
      VP
  ____|___
 |        NP
 |     ___|___
 VP  Det      NP
 |    |       |
book that    flight

[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\admin\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

Code & Output:

```
import nltk
from nltk.tokenize import word_tokenize
nltk.download('punkt_tab')

print("Karthik Vinod, TCS2627033")

words=word_tokenize(text)
grammar = """
S -> NP VP
NP -> Pronoun
Pronoun -> 'He'
VP -> V NP
V -> 'likes'
NP -> N
N -> 'music'
"""

sentence = "He likes music"
cfg=nltk.CFG.fromstring(grammar)
words=word_tokenize(sentence)
parser=nltk.ChartParser(cfg)
print("Parse Tree:")
for tree in parser.parse(words):
    print(tree)
tree.pretty_print()
```

```
Karthik Vinod, TCS2627033
Parse Tree:
(S (NP (Pronoun He)) (VP (V likes) (NP (N music))))
          S
     _____|___
    |         VP
    |      ___|___
    NP     |      NP
    |      |      |
Pronoun    V      N
    |      |      |
    He   likes  music

[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\Karthik\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

Code & Output:

```
import nltk
from nltk.tokenize import word_tokenize
nltk.download('punkt_tab')

print("Karthik Vinod, TCS2627033")

words=word_tokenize(text)
words=word_tokenize(text)
grammar = """
S -> NP VP
NP -> 'John'
VP -> V NP
V -> 'eats'
NP -> Det N
Det -> 'an'
N -> 'apple'
"""

sentence="John eats an apple"
cfg=nltk.CFG.fromstring(grammar)
words=word_tokenize(sentence)
parser=nltk.ChartParser(cfg)
print("Parse Tree:")
for tree in parser.parse(words):
    print(tree)
tree.pretty_print()
```

```
Karthik Vinod, TCS2627033
Parse Tree:
(S (NP John) (VP (V eats) (NP (Det an) (N apple))))
           S
  _________|___
 |             VP
 |     ________|___
 |    |            NP
 |    |         ___|____
 NP   V       Det       N
 |    |        |        |
John eats      an     apple

[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\Karthik\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

NLP:

Practical No 10:

Name: Karthik Vinod

Roll No: TCS2627033

Aim: Write a program to implement Parts of Speech Tagging using spacy.

Python Libraries & Modules:

- **pip install spacy:** Installs the spaCy library, which is used for Natural Language Processing (NLP) tasks such as tokenization, POS tagging, named entity recognition, and text analysis.
- **import spacy:** Imports the spaCy library into the Python program so its NLP functions and tools can be used.
- **spacy.cli.download("en_core_web_sm"):** Downloads the English small language model (en_core_web_sm) required by spaCy to analyze English text.
- **nlp = spacy.load("en_core_web_sm"):** Loads the downloaded English language model and stores it in the nlp variable for processing text.

Text Processing:

- **text = input("Enter the Sentence: "**): Takes a sentence from the user as input.
- **doc = nlp(text):** Processes the input text using the spaCy NLP model and creates a Doc object containing the analyzed text.
- **for word in doc:**: Loops through each word (token) in the processed document.
- **word.text:** Returns the actual word or token from the sentence.
- **word.pos_:** Returns the Part-of-Speech (POS) category of the word, such as noun, verb, adjective, or preposition.
- **word.tag_:** Returns the detailed grammatical tag assigned to the word, such as NN, VB, PRP, or IN.
- **spacy.explain(word.tag_):** Provides a human-readable explanation of the grammatical tag.
- **print(f"{word.text} ---> {word.pos_} {word.tag_} {spacy.explain(word.tag_)}"):** Displays each word along with its POS category, detailed grammatical tag, and explanation.

Code:

```
pip install spacy
```

```
Requirement already satisfied: spacy in /usr/local/lib/python3.13/dist-packages (3.8.16)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.13/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.0.5)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.0.15)
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.0.13)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.13/dist-packages (from spacy) (3.0.13)
Requirement already satisfied: thinc<8.4.0,>=8.3.12 in /usr/local/lib/python3.13/dist-packages (from spacy) (8.3.13)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.5.3 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.5.3)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.0.10)
Requirement already satisfied: weasel<2.0.0,>=1.0.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.0.0)
Requirement already satisfied: confection<2.0.0,>=1.3.2 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.3.3)
Requirement already satisfied: typer<1.0.0,>=0.3.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (0.27.2)
```

```
import spacy
spacy.cli.download("en_core_web_sm")

nlp=spacy.load("en_core_web_sm")
text=input("Enter the Senetnce: ")
doc=nlp(text)
for word in doc:
  print(f"{word.text} ---> {word.pos_} {word.tag_}
{spacy.explain(word.tag_)}")

#I will google about facebook
#To the left of the room
#I left the room
```

Output:

Input 1: I will google about facebook.

```
Enter the Senetnce: I will google about facebook
I ---> PRON PRP pronoun, personal
will ---> AUX MD verb, modal auxiliary
google ---> VERB VB verb, base form
about ---> ADP IN conjunction, subordinating or preposition
facebook ---> PROPN NNP noun, proper singular
```

Input 2: To the left of the room.

```
Enter the Senetnce: To the left of the room
To ---> ADP IN conjunction, subordinating or preposition
the ---> DET DT determiner
left ---> NOUN NN noun, singular or mass
of ---> ADP IN conjunction, subordinating or preposition
the ---> DET DT determiner
room ---> NOUN NN noun, singular or mass
```

Input 3: I left the room.

```
Enter the Senetnce: I left the room
I ---> PRON PRP pronoun, personal
left ---> VERB VBD verb, past tense
the ---> DET DT determiner
room ---> NOUN NN noun, singular or mass
```

NLP:

Practical No 11:

Name: Karthik Vinod

Roll No: TCS2627033

Aim: Write a program to implement Named Entity Recognition using spacy.

Python Libraries & Modules:

- **import spacy:** Imports the spaCy library, which is used for Natural Language Processing (NLP) tasks such as tokenization, Part-of-Speech (POS) tagging, Named Entity Recognition (NER), and text analysis.
- **spacy.cli.download("en_core_web_sm"):** Downloads the English small language model used by spaCy to analyze and understand English text.
- **nlp = spacy.load("en_core_web_sm"):** Loads the English language model into the nlp variable so that it can be used to process the given text.

Text Processing:

- **content = """..."""":** Stores the given article or paragraph as a multi-line string in the content variable.
- **doc = nlp(content):** Processes the content using the spaCy NLP model and creates a Doc object containing tokens and linguistic information about the text.

Named Entity Recognition (NER)

- **for ent in doc.ents:**: Loops through all the named entities identified by spaCy in the document.
- **ent.text:** Returns the actual text of the identified entity.
- **Example:** Nvidia, Amazon Web Services, Microsoft Azure.
- **ent.start_char:** Returns the starting character position of the entity in the original text.
- **ent.end_char:** Returns the ending character position of the entity in the original text.
- **ent.label_:** Returns the entity label assigned by spaCy, such as PERSON, ORG, GPE, DATE, or MONEY.
- **print(ent.text, ent.start_char, ent.end_char, ent.label_):** Displays the entity, its starting and ending character positions, and its entity type.

Entity Visualization:

- **from spacy import displacy:** Imports displacy, spaCy's built-in visualization tool used to display linguistic information in a graphical format.
- **displacy.render(doc, style="ent", jupyter=True):** Displays the recognized named entities in the text with different labels and highlighting. The style="ent" option specifies entity visualization, while jupyter=True allows the visualization to be displayed directly in Jupyter Notebook or Google Colab.

Code:

```
import spacy
spacy.cli.download("en_core_web_sm")
nlp=spacy.load("en_core_web_sm")

content="""Nvidia's (NASDAQ: NVDA) second-quarter 2027 (ended July 26,
2026) earnings report made the usual point in unusually large numbers.
Data center revenue reached $89 billion during the quarter, up 18% from
the prior quarter and 117% year over year. What I think matters more
than the headline figure is the bifurcation inside that number.
Hyperscale customers generated $48.7 billion in revenue, up 13%
sequentially and 102% from a year ago. The AI Clouds, Industrial, and
Enterprise group (ACIE) generated $40.3 billion, an increase of 25%
from last quarter and 138% year over year.
Missed Nvidia in 2009? This Rare Signal Is Flashing Again. In 2009, a
"Double Down" signal flashed for a little-known chipmaker called
Nvidia. For the first time in years, that same "Total Conviction"
signal is flashing for a company 1/100th the size of Nvidia.
While big tech is still the largest buyer of Nvidia's systems, it is no
longer the fastest growing. That shift changes how the AI story should
be read.
The hyperscalers form Nvidia's foundation
The hyperscale bucket primarily revolves around the public cloud
oligopoly: Amazon Web Services (AWS), Microsoft Azure, and Google Cloud
Platform (GCP). These companies aren't buying Nvidia's products as a
hobby. They buy them because GPU clusters have become a scarce resource
supporting training runs, large-scale inference deployments, and the
rented capacity that developers consume.
Despite designing its own Trainium, Inferentia, and Graviton silicon,
AWS remains one of Nvidia's largest customers. The reason is simple:
Nvidia's GPUs and CPUs play a critical role in how start-ups, labs, and
enterprises rent Blackwell and Rubin chips from cloud providers without
owning their own facilities packed with liquid-cooled racks.
Azure has bound itself to this same stack. Microsoft's Copilot suite,
OpenAI-related training, and Azure-OpenAI services all rely on Nvidia
systems. This structure is unique, as it makes Microsoft both a
customer of and a distribution channel for Nvidia
```

```
Google Cloud also designs its own custom silicon, called Tensor
Processing Units (TPUs). Even so, Alphabet still buys enormous
quantities of Nvidia hardware. One reason why is that many customers
want access to CUDA, Nvidia's software ecosystem that runs on top of
its GPUs. This makes the ability to move AI workloads across clouds
much more efficient. For now, Google's TPUs cannot serve that type of
demand on its own.
Taken together, these three cloud hyperscalers form the floor
supporting Nvidia's data center operation. By signing multiyear
capacity plans they essentially turn Nvidia's racks into an
infrastructure-as-a-service empire. When revenue from the hyperscalers
more than doubles in a year, it is evidence that the largest buyers see
returns on procuring more accelerators. It is not evidence, however,
that the GPU market only has three buyers. """
doc=nlp(content)

for ent in doc.ents:
  print(ent.text, ent.start_char, ent.end_char, ent.label_)
from spacy import displacy
displacy.render(doc, style="ent", jupyter=True)
```

Output:

```
Nvidia 0 6 GPE
NVDA 18 22 ORG
second-quarter 2027 24 43 DATE
July 26, 2026 51 64 DATE
$89 billion 159 170 MONEY
the quarter 178 189 DATE
18% 194 197 PERCENT
the prior quarter 203 220 DATE
117% 225 229 PERCENT
$48.7 billion 367 380 MONEY
13% 396 399 PERCENT
102% 417 421 PERCENT
a year ago 427 437 DATE
The AI Clouds 439 452 ORG
ACIE 488 492 ORG
$40.3 billion 504 517 MONEY
25% 534 537 PERCENT
last quarter 543 555 DATE
138% 560 564 PERCENT
Nvidia 588 594 ORG
2009 598 602 DATE
This Rare Signal Is Flashing Again 604 638 ORG
2009 643 647 DATE
Nvidia 716 722 GPE
first 732 737 ORDINAL
years 746 751 DATE
Total Conviction 764 780 WORK_OF_ART
Nvidia 835 841 GPE
Nvidia 888 894 GPE
AI 970 972 ORG
```



NLP:

Practical No 12:

Name: Karthik Vinod

Roll No: TCS2627033

Aim: Write a program to implement Text to Speech Using gTTS (Google Text to Speech).

Python Libraries & Modules:

- **!pip install gTTS:** Installs the gTTS (Google Text-to-Speech) library, which converts written text into spoken audio.
- **from gtts import gTTS:** Imports the gTTS class from the gTTS library, which is used to convert text into speech.
- **import os:** Imports Python's built-in OS module, which provides functions for interacting with the operating system, such as managing files and folders.

Text-to-Speech Processing:

- **language = "en":** Specifies English (en) as the language for the generated speech.
- **text = input("Enter a Sentence: "):** Takes a sentence from the user as input, which will be converted into speech.
- **myobj = gTTS(text, lang=language, slow=False):** Creates a gTTS object that converts the entered text into speech. lang specifies the language, while slow=False makes the speech play at the normal speed.
- **myobj.save("output.mp3"):** Saves the generated speech as an MP3 audio file named output.mp3.

Code:

```
!pip install gTTS
```

```
Requirement already satisfied: gTTS in /usr/local/lib/python3.13/dist-packages (2.5.4)
Requirement already satisfied: requests<3,>=2.27 in /usr/local/lib/python3.13/dist-packages (from gTTS) (2.32.4)
Collecting click<8.2,>=7.1 (from gTTS)
  Using cached click-8.1.8-py3-none-any.whl.metadata (2.3 kB)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests<3,>=2.27->gTTS) (3.4.9)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests<3,>=2.27->gTTS) (3.19)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests<3,>=2.27->gTTS) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests<3,>=2.27->gTTS) (2026.7.22)
```

```
from gtts import gTTs
import os
language="en"
text=input("Enter a Sentence: ")
myobj=gTTS(text,lang=language, slow=False)
myobj.save("output1.mp3")
```

Output:

```
Enter a Sentence: Hi Myself Karthik Vinod, My roll number is TCS2627033
```



